Case Study Report

on

Case Study 143:

Project:- Smart Waste Collection & Recycling Management System

Student Details: Name: Ankitraj Jha

Student ID: 150096725050

College: ITM Skills University, Kharghar

Batch: 2025-2029

Course: BTech (CSE)

Subject: DSA-1 in C++

Case Study: Smart Waste Collection & Recycling Management System

Github Link: https://github.com/Ankitraj-17/DSA-Final

Index

---

1. Introduction to the Case Study

As modern municipalities undergo rapid urbanization, the volume of solid waste generated daily escalates exponentially. Municipal corporations are tasked with coordinating collection routines under constraints of restricted budgets, limited fleets, fuel cost fluctuations, and environmental goals.

Traditional municipal solid waste management systems suffer from high operational overhead due to fixed schedules and static routing. Collection vehicles visit zones without knowing bin fill status, leading to empty trips or overflows. Furthermore, citizens complain of long response times for service requests, and managing databases of thousands of bins becomes slow if search indexes degrade.

This case study designs and implements a **Smart Waste Collection & Recycling Management System** in C++ using custom-designed Data Structures and Algorithms. The platform integrates IoT sensor simulation, shortest-path road navigation, dynamic priority queues, and self-balancing tree databases to provide a highly scalable, real-time command dashboard.

Efficient algorithmic waste management is applicable in:

Smart Cities: Municipal route optimization and sensor-driven garbage collection.
Logistics & Fleet Management: Real-time asset checks, license lookups, and scheduling.
Environmental Sustainability: Dynamic estimation of recyclable volumes and resource deployment.

---

2. Problem Statement (Abstract)

1. Municipal corporations face significant logistical hurdles in managing urban waste streams, necessitating a centralized platform to coordinate collection, recycling, and fleet monitoring while advancing environmental sustainability goals.

2. Technical Implementation Strategy:
    - Sorting Algorithms: Employed to rank and prioritize waste collection schedules based on bin fill levels and urgency.
    - Searching Algorithms: Utilized for high-speed retrieval of historical collection and recycling data records.
    - Stack Structures: Implemented to manage operational recovery procedures, providing LIFO (Last-In-First-Out) functionality for undoing recent system modifications.
    - Queue Management: Dedicated to processing citizen service requests in a fair, sequential order.
    - BST & AVL Trees: Used to maintain organized, self-balancing databases of waste management assets

    - BFS, DFS & Dijkstra's Algorithm: Applied to analyze city infrastructure and compute the most cost-effective collection routes.

    - Hashing Techniques: Integrated to enable near-instantaneous identification of vehicle and bin assets through polynomial mapping.

3. Product Building Objectives: The development of an integrated waste management dashboard is central to the project, providing real-time visibility into collection schedules, recycling metrics, GPS vehicle tracking, and comprehensive environmental impact reports.
4. Strategic Operational Priorities:
    Prioritization of high-waste generation zones to prevent overflows.
    Optimization of vehicle utilization and reduction of fuel-related collection costs.
    Improvement of recycling efficiency through data-driven resource deployment.
    Enhancement of long-term environmental sustainability within the municipality.

Design / Architecture

System Overview

The Smart Waste Collection & Recycling Management System is designed as a modular, menu-driven application that simulates the workflow of a real municipal waste administration

system. The architecture integrates multiple Data Structures and Algorithms to efficiently manage bin records, collection schedules, vehicle tracking, and route optimization.

The system follows a layered architecture where user requests are processed through specialized modules, each responsible for a specific municipal operation.

Architectural Components

1. User Interaction Layer

The User Interaction Layer acts as the interface between municipal administrators and the system. It provides menu-driven operations through which users can:

- Register new waste bins
- Search existing bins
- Schedule collection priorities
- Process citizen requests
- Manage vehicle registries
- Optimize collection routes
- View environmental reports
- Analyze technical metrics

All user inputs are validated before being forwarded to the corresponding processing modules.

2. Waste Bin Management Module

This module is responsible for maintaining all waste bin records.
Each bin contains:

- Bin ID
- Location
- Fill Level (%)
- Waste Type
- Capacity (Liters)

Bins are stored inside an AVL Tree as the primary database, ensuring balanced and rapid access, and a BST as backup storage.

Responsibilities

- Register new bins
- Update bin information
- Store bin metadata
- Support searching and reporting operations

Data Structure Used
AVL Tree & Binary Search Tree (BST)

Reason for Selection

Trees provide efficient hierarchical storage for searching and sorting records. AVL Trees ensure O(log N) operations by remaining strictly balanced.

3. Citizen Service Requests Module

When a new service request is submitted, it enters the request queue before being processed.
The queue follows the First-In-First-Out (FIFO) principle.

Workflow
Request A Submitted
Request B Submitted
Request C Submitted

Processing Order:
A → B → C

Responsibilities
- Maintain request order
- Ensure fairness in processing
- Handle pending complaints

Data Structure Used
Queue

Reason for Selection
Citizen requests are naturally processed in the order they are received. Queue provides an efficient and realistic representation of this workflow.

4. Collection Planning Module

Certain bins require immediate attention based on their fill level and capacity.
Examples:
- Bins > 80% full
- Hazardous waste bins

Such bins are assigned higher priority scores and sorted using Quick Sort.

Responsibilities
- Prioritize urgent collections
- Generate prioritized collection schedules
- Improve municipal response time

Algorithm Used
Quick Sort

Reason for Selection

Quick Sort allows efficient sorting of the collection priority list in O(N log N) average time.

5. Search and Retrieval Module

Municipalities frequently require retrieval of bin records.
To improve search efficiency, bins are sorted according to their IDs and searched using Binary Search.

Workflow
Sorted IDs: 101 102 103 104 105 106

Search 105
Step 1: Mid = 103
Step 2: Move Right
Step 3: Found 105

Responsibilities

- Fast bin retrieval
- Reduce search time
- Improve user productivity

Algorithm Used
Binary Search

6. Route Planning Module

City zones and their road connections form a City Network.
These relationships are represented using a City Graph.

Responsibilities

- Store zone connections
- Analyze network reachability
- Find shortest path for collection trucks

Data Structure Used
Directed Graph (Adjacency List)

Reason for Selection
Graphs naturally model city maps where intersections are nodes and roads are edges.

7. Route Optimization Module

Finding the fastest way to travel between zones is crucial for fuel efficiency.

Responsibilities

- Compute shortest routes
- Optimize travel time

- Support municipal logistics

Algorithm Used
Dijkstra's Algorithm

8. Operational Recovery Module

Municipal administrators might need to review or undo recent system actions.

Responsibilities
- Track recent user actions
- View log history
- Revert last entry

Data Structure Used
Stack

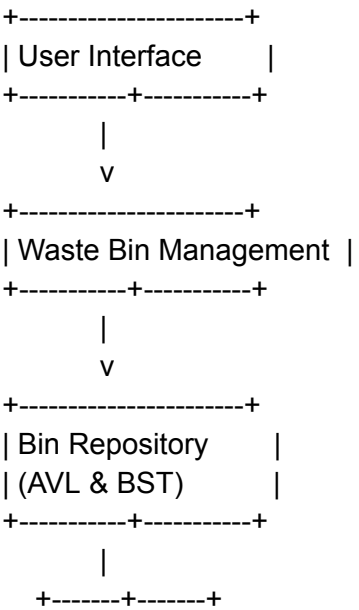
9. Vehicle Management Module

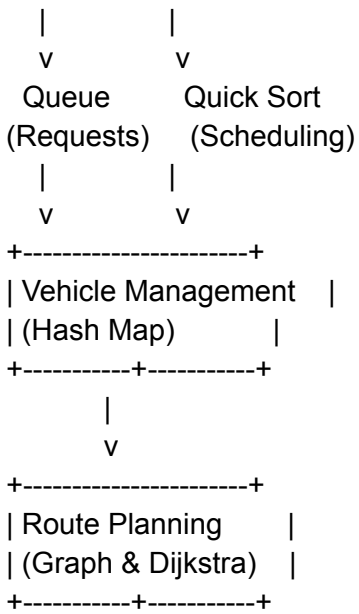
Municipalities frequently require retrieval of vehicle records by license plate.
To improve search efficiency, vehicles are stored in a Hash Table with chaining.

Responsibilities
- Fast vehicle retrieval
- Reduce search time
- Handle vehicle registration

Data Structure Used
Hash Map (Chaining)

Complete System Architecture Diagram





Algorithm Description

Algorithm 1: Bin Registration
Objective: Add a new waste bin into the system.
Steps:

- Read bin details.
- Create a WasteBin object.
- Insert bin into AVL Tree.
- Insert bin into BST Tree.
- Display success message.
- Push action to Stack.

Pseudocode:
INPUT WasteBin
Insert into AVL_Tree
Insert into BST_Tree
Push to Action_Stack
Display Success

Complexity:
Time Complexity: O(log N)
Space Complexity: O(1)

Algorithm 2: Queue Processing
Objective: Process citizen requests according to filing order.
Steps:

1. Check whether queue is empty.
2. Remove front request.
3. Display processed request.
4. Push action to Stack.

Pseudocode:
IF Queue Empty
Stop
ELSE
Remove Front Element
END IF
Complexity:
Time Complexity: O(1)
Space Complexity: O(1)

Algorithm 3: Collection Scheduling
Objective: Sort bins by priority score for collection.
Steps:

1. Retrieve all bins.
2. Partition array around a pivot.
3. Recursively sort sub-arrays.
4. Display sorted collection list.

Pseudocode:
Function QuickSort(Arr, Low, High)
IF Low < High
Pivot = Partition(Arr, Low, High)
QuickSort(Arr, Low, Pivot - 1)
QuickSort(Arr, Pivot + 1, High)
END IF
Complexity:
Time Complexity: O(N log N)
Space Complexity: O(log N)

Algorithm 4: Shortest Route Planning
Objective: Find the shortest path between two city zones.
Steps:

1. Initialize distances to INF.
2. Set source distance to 0.
3. Extract minimum distance node.
4. Update neighbor distances.
5. Repeat until the destination is reached.

Complexity:
Time Complexity: O(V^2 + E)
Space Complexity: O(V)

Algorithm 5: Vehicle Hash Search
Objective: Retrieve a vehicle record quickly.
Steps:

1. Compute hash value for plate number.
2. Go to the index in the hash table.
3. Traverse linked list (chain) if collision.
4. Return vehicle details.

Complexity:

Time Complexity: O(1) Average
Space Complexity: O(1)

Algorithm 6: Network Coverage Tracking
Objective: Trace connected city zones for service coverage.
Steps:

1. Start from the selected zone.
2. Mark node as visited.
3. Visit connected road nodes.
4. Continue recursively.
5. Print traversal path.

DFS Traversal Example: Central -> North -> West -> South
Complexity:
Time Complexity: O(V + E)
Space Complexity: O(V)

Algorithm 7: Operational Recovery
Objective: Recover and undo recent system actions.
Steps:

1. Check if the stack is empty.
2. Pop top action from stack.
3. If action matches "Deleted":
    a. Restore items.
4. Otherwise mark as undone.
5. Update system state.

Example: Add Bin 101 -> Remove Bin 102 -> Action Undone.
Complexity:
Time Complexity: O(1)
Space Complexity: O(1)

Complexity Analysis

Operation	**Algorithm / Data Structure**	**Time Complexity**	**Space Complexity**
Register Bin	AVL Tree / BST Insert	O(log N)	O(1)
Process Citizen Request	Queue Dequeue	O(1)	O(1)
Collection Scheduling	Quick Sort	O(N log N)	O(log N)
Search Bin by ID	Binary Search	O(log N)	O(1)
Search Bin by Location	Linear Search	O(N)	O(1)
Add Route Connection	Graph Edge Insertion	O(1)	O(1)

Operation	Algorithm / Data Structure	Time Complexity	Space Complexity
Find Shortest Route	Dijkstra's Algorithm	O(V^2 + E)	O(V)
Check Network Coverage	Depth First Search (DFS)	O(V + E)	O(V)
Check Reachable Areas	Breadth First Search (BFS)	O(V + E)	O(V)
Retrieve Vehicle	Hash Map Search	O(1) Avg	O(1)
Operational Recovery	Stack Push/Pop	O(1)	O(1)

Where:

N = Total number of bins stored in the system
V = Number of vertices (city zones) in the graph
E = Number of edges (road connections) in the graph

4. Methods & Algorithms Technology Applied

Feature	Data Structure / Algorithm	Justification
Collection Scheduling	**Sorting Algorithms**	Orders the collection zones by waste generation level to prioritize high-need areas efficiently.
Record Retrieval	**Searching Algorithms (Binary Search)**	Allows quick, logarithmic-time lookup of historical recycling and collection records from sorted databases.
Recovery Procedures	**Stack**	Uses LIFO (Last-In-First-Out) ordering. The most recent operational change is at the top, perfectly mimicking the behavior of an 'Undo' button.
Service Requests	**Queue**	Enforces FIFO (First-In-First-Out) processing, guaranteeing that citizen or municipal requests are processed strictly in the order they were submitted.
Asset Identification	**Hash Table**	Provides near-instant (O(1)) lookup times to verify if a given physical asset

Feature	Data Structure / Algorithm	Justification
		(RFID or GPS ID) already exists or to find its status.
Waste Database	**BST / AVL Trees**	Automatically balances the collection database so that insertion, deletion, and searching remain fast and predictable at O(log N).
Route Optimization	**Graph & Dijkstra's Algorithm**	Models city intersections as vertices and roads as edges with distance weights. Dijkstra guarantees the shortest and most cost-effective path for garbage trucks.

5. Problem Statement / Case Study Implementation Details and Snapshots

The core logic of the system is written in `smart_waste_system.cpp`. The platform avoids pre-built standard templates, creating custom objects for the data layers.

Core Structure Implementations

1. Custom Singly-Linked Stack (LIFO Recovery Log)

The recovery mechanism logs all operational modifications using dynamic singly linked nodes:

2. Custom FIFO Queue (Service Request Manager)

Citizen tickets are processed in order using a linked queue structure tracking head and tail nodes:

3. Custom AVL Self-Balancing Tree (Balanced Database Storage)

Primary database storage inherits standard binary search operations but wraps them in rotations to maintain logarithmic height:

```
None
struct TreeNode {
    WasteBin bin;
    TreeNode *left, *right;
    int height;
```

```
};

class AVL {
public:
    TreeNode* root = nullptr;
    int count = 0;

    int getH(TreeNode* n) { return n ? n->height : 0; }

    int getBF(TreeNode* n) {
        return n ? getH(n->left) - getH(n->right) : 0;
    }

    void updH(TreeNode* n) {
        if (n) n->height = 1 + maxVal(getH(n->left),
getH(n->right));
    }

    TreeNode* rotR(TreeNode* y) {
        TreeNode* x = y->left, *T2 = x->right;
        x->right = y;
        y->left = T2;
        updH(y);
        updH(x);
        return x;
    }

    TreeNode* rotL(TreeNode* x) {
        TreeNode* y = x->right, *T2 = y->left;
        y->left = x;
        x->right = T2;
        updH(x);
        updH(y);
        return y;
    }

    TreeNode* bal(TreeNode* n) {
        updH(n);
        int bf = getBF(n);
```

```
        if (bf > 1) {
            if (getBF(n->left) < 0) n->left = rotL(n->left);
            return rotR(n);
        }
        if (bf < -1) {
            if (getBF(n->right) > 0) n->right = rotR(n->right);
            return rotL(n);
        }
        return n;
    }

    TreeNode* ins(TreeNode* n, WasteBin b) {
        if (!n) {
            count++;
            return new TreeNode{b, nullptr, nullptr, 1};
        }
        if (b.id < n->bin.id) n->left = ins(n->left, b);
        else if (b.id > n->bin.id) n->right = ins(n->right, b);
        else return n;
        return bal(n);
    }

    void insert(WasteBin b) { root = ins(root, b); }
};
```

4. Custom Hash Table with Collision Chaining (License lookup)

Fleet assets are indexed in a size-53 bucket structure using a polynomial rolling hash:

5. Shortest Route Optimization (Dijkstra's Pathfinder)

Computes shortest paths on the city graph using relaxation over arrays:

None

```
void dijkstra(int src, int dst) {
    int dist[maxVertices], prev[maxVertices];
    bool visited[maxVertices];

    for (int i = 0; i < V; i++) {
        dist[i] = 999999;
```

```
        prev[i] = -1;
        visited[i] = false;
    }

    dist[src] = 0;

    for (int count = 0; count < V - 1; count++) {
        int minDist = 999999, u = -1;
        for (int v = 0; v < V; v++) {
            if (!visited[v] && dist[v] < minDist) {
                minDist = dist[v];
                u = v;
            }
        }

        if (u == -1) break;
        visited[u] = true;

        for (size_t i = 0; i < adjList[u].size(); i++) {
            int neighbor = adjList[u][i].dest;
            int weight = adjList[u][i].weight;
            if (!visited[neighbor] && dist[u] + weight <
dist[neighbor]) {
                dist[neighbor] = dist[u] + weight;
                prev[neighbor] = u;
            }
        }
    }
}
```

5.1. SnapShorts

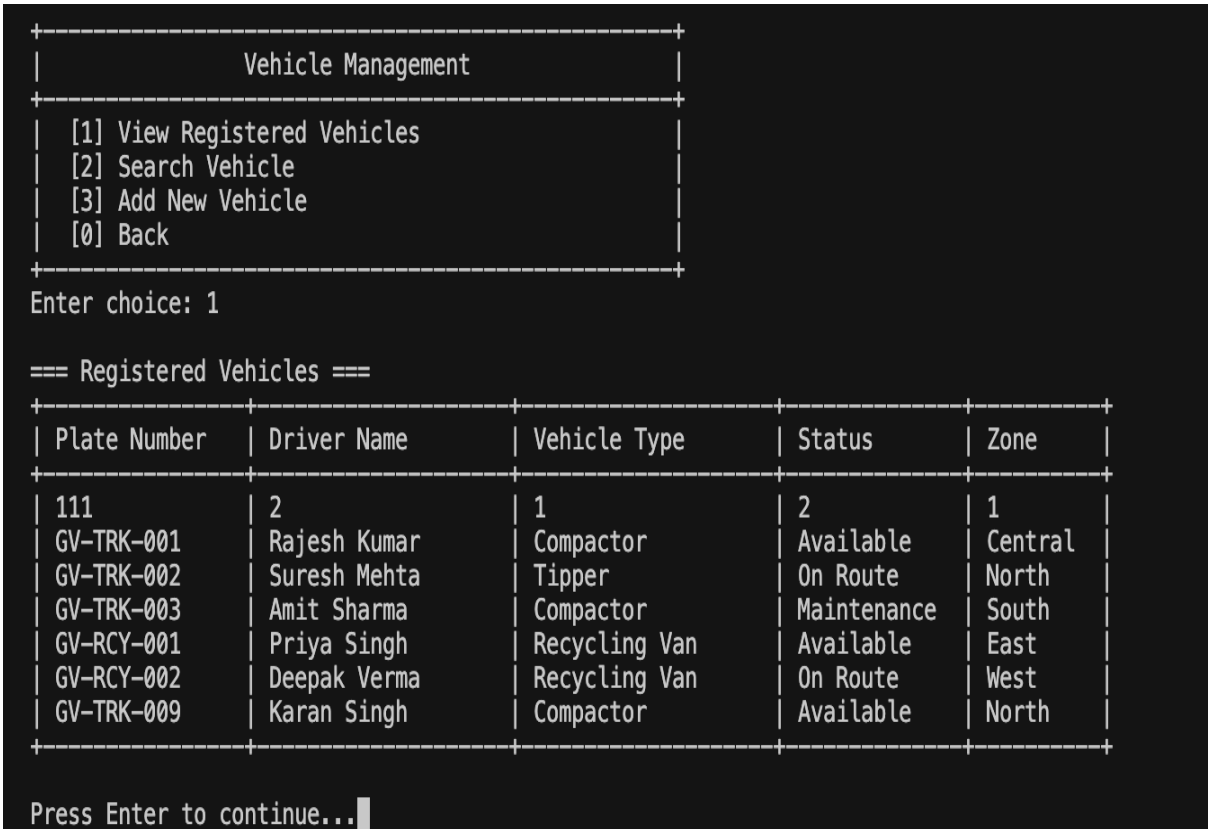
1. **Main Dashboard**

```
+-----------------------------------------------------------+
|  Smart Waste Collection & Recycling Management System     |
+-----------------------------------------------------------+
|  System Status                                            |
|    Total Bins          : 11                               |
|    Urgent Bins         : 4 need collection                |
|    Pending Requests    : 5 citizen issues                 |
|    Registered Vehicles : 7                                |
|    Recommended Action  : Review Collection Planning       |
+-----------------------------------------------------------+
|   [1] Waste Bin Management                                |
|   [2] Collection Planning                                 |
|   [3] Citizen Service Requests                            |
|   [4] Vehicle Management                                  |
|   [5] Route Planning                                      |
|   [6] Environmental Report                                |
|   [7] Technical Analysis                                  |
|   [8] Recovery Log                                        |
|   [0] Exit                                                |
+-----------------------------------------------------------+
Enter Choice:
```

2. Waste Bin Management

```
+-----------------------------------------------+
|              Waste Bin Management             |
+-----------------------------------------------+
|  [1] View All Bins                            |
|  [2] Add New Bin                              |
|  [3] Search Bin by ID                         |
|  [4] Search Bin by Location                   |
|  [5] Remove Bin                               |
|  [0] Back                                     |
+-----------------------------------------------+
Enter choice: 1

=== Waste Bin Records ===
+--------+-----------------------+--------+----------+-------------+-----------+
| ID     | Location              | Fill%  | Capacity | Type        | Status    |
+--------+-----------------------+--------+----------+-------------+-----------+
| 2      | 101                   | 2%     | 3L       | 6           | Normal    |
| 101    | Main Street           | 92%    | 500L     | Organic     | URGENT    |
| 103    | Industrial Area       | 88%    | 1000L    | General     | URGENT    |
| 104    | Residential North     | 30%    | 400L     | Organic     | Normal    |
| 105    | Residential South     | 76%    | 400L     | Recyclable  | Moderate  |
| 106    | Market District       | 95%    | 600L     | General     | URGENT    |
| 107    | Commercial Hub        | 55%    | 750L     | Hazardous   | Moderate  |
| 108    | University Area       | 40%    | 350L     | Recyclable  | Normal    |
| 109    | Town Square           | 83%    | 450L     | Organic     | URGENT    |
| 110    | Riverside Zone        | 62%    | 550L     | General     | Moderate  |
| 1003   | mumbai                | 50%    | 7L       | wet         | Moderate  |
+--------+-----------------------+--------+----------+-------------+-----------+
Total Bins: 11 | Primary Storage Depth: 4

Press Enter to continue...
```

3. Collection Planning



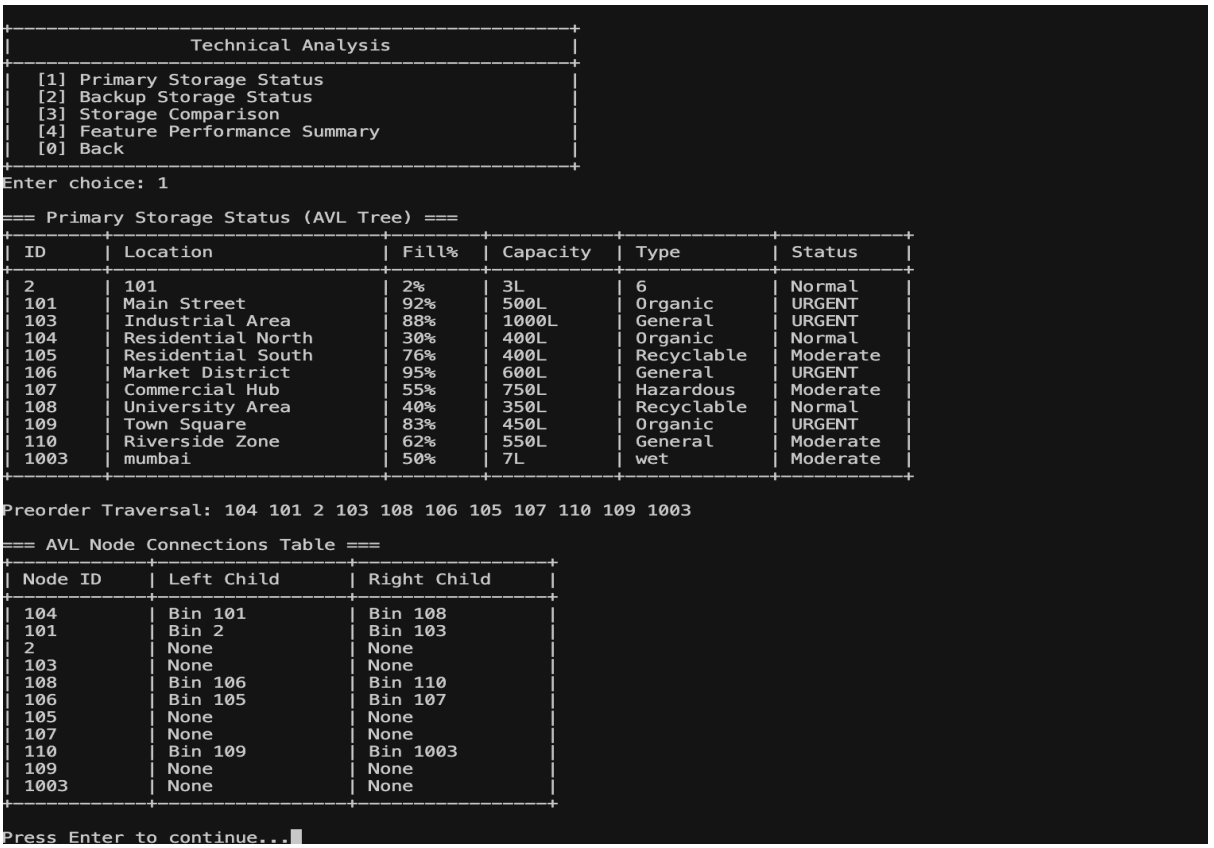
4. Route Planning

```
Enter choice: 3

+-------------------------------------------------+
|                 Route Planning                  |
+-------------------------------------------------+
|  [1] Show City Map                              |
|  [2] View Reachable Service Areas               |
|  [3] Check City Network Coverage                |
|  [4] Find Shortest Collection Route             |
|  [0] Back                                       |
+-------------------------------------------------+
Enter choice: 1

=== City Connections Map ===
+--------------------+----------------------------------------------------+
| Source Area        | Connected Areas (Distance)                         |
+--------------------+----------------------------------------------------+
| Central Depot      | Main Street (2km), Park Avenue (3km)               |
| Main Street        | Central Depot (2km), Industrial Area (4km), Reside |
| Park Avenue        | Central Depot (3km), Residential South (3km), Mark |
| Industrial Area    | Main Street (4km), Commercial Hub (1km)            |
| Residential North  | Main Street (2km), Commercial Hub (3km)            |
| Residential South  | Park Avenue (3km), University Area (2km)           |
| Market District    | Park Avenue (5km), Recycling Center (4km)          |
| Commercial Hub     | Industrial Area (1km), Residential North (3km), Un |
| University Area    | Residential South (2km), Commercial Hub (6km), Rec |
| Recycling Center   | Market District (4km), University Area (3km)       |
+--------------------+----------------------------------------------------+

Press Enter to continue...
```

6. Environment Report

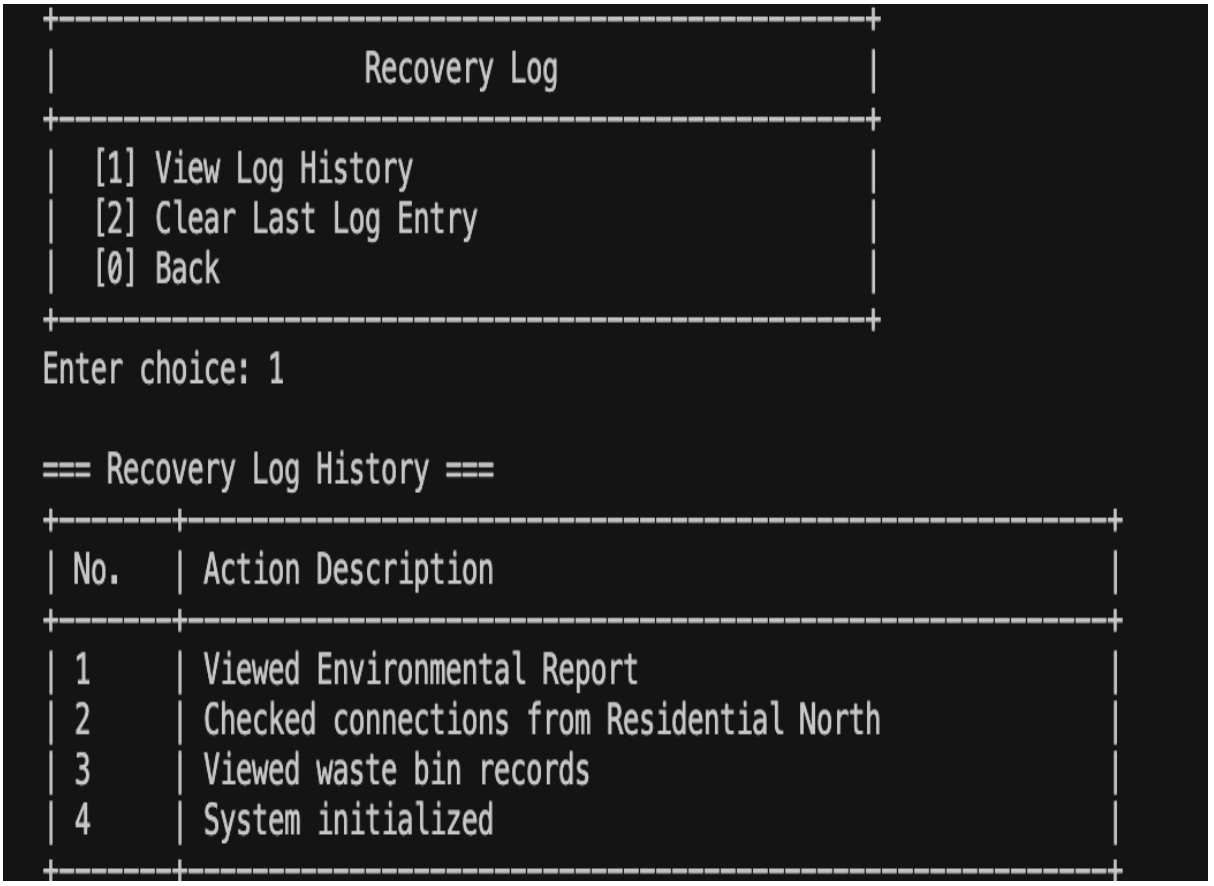
```
=== Environmental Report ===
+--------------------------------------+----------------+
| Metric                               | Value          |
+--------------------------------------+----------------+
| Total Bins Monitored                 | 11             |
| Bins Needing Collection (>=80%)      | 4              |
| Estimated Waste Volume               | 3609 L         |
| Recyclable Waste Share               | 12.3%          |
| Recycling Share Progress             | [=             |
| Pending Citizen Issues               | 5              |
| Registered Vehicles                  | 7              |
+--------------------------------------+----------------+

Press Enter to continue...
```

7. Technical Analysis



```
+-------------------------------------------------+
|                Technical Analysis               |
+-------------------------------------------------+
|  [1] Primary Storage Status                     |
|  [2] Backup Storage Status                      |
|  [3] Storage Comparison                         |
|  [4] Feature Performance Summary                |
|  [0] Back                                       |
+-------------------------------------------------+
Enter choice: 1

=== Primary Storage Status (AVL Tree) ===
+--------+-------------------------+--------+------------+--------------+------------+
| ID     | Location                | Fill%  | Capacity   | Type         | Status     |
+--------+-------------------------+--------+------------+--------------+------------+
| 2      | 101                     | 2%     | 3L         | 6            | Normal     |
| 101    | Main Street             | 92%    | 500L       | Organic      | URGENT     |
| 103    | Industrial Area         | 88%    | 1000L      | General      | URGENT     |
| 104    | Residential North       | 30%    | 400L       | Organic      | Normal     |
| 105    | Residential South       | 76%    | 400L       | Recyclable   | Moderate   |
| 106    | Market District         | 95%    | 600L       | General      | URGENT     |
| 107    | Commercial Hub          | 55%    | 750L       | Hazardous    | Moderate   |
| 108    | University Area         | 40%    | 350L       | Recyclable   | Normal     |
| 109    | Town Square             | 83%    | 450L       | Organic      | URGENT     |
| 110    | Riverside Zone          | 62%    | 550L       | General      | Moderate   |
| 1003   | mumbai                  | 50%    | 7L         | wet          | Moderate   |
+--------+-------------------------+--------+------------+--------------+------------+

Preorder Traversal: 104 101 2 103 108 106 105 107 110 109 1003

=== AVL Node Connections Table ===
+------------+------------------+------------------+
| Node ID    | Left Child       | Right Child      |
+------------+------------------+------------------+
| 104        | Bin 101          | Bin 108          |
| 101        | Bin 2            | Bin 103          |
| 2          | None             | None             |
| 103        | None             | None             |
| 108        | Bin 106          | Bin 110          |
| 106        | Bin 105          | Bin 107          |
| 105        | None             | None             |
| 107        | None             | None             |
| 110        | Bin 109          | Bin 1003         |
| 109        | None             | None             |
| 1003       | None             | None             |
+------------+------------------+------------------+

Press Enter to continue...
```

8. Recovery Log



6. Case Study Results and Conclusion

Analysis of Results

1. Database Performance:
   The AVL Tree maintained a balanced structure while storing waste bin records, resulting in efficient search, insertion, and deletion operations with O(log N) complexity. Compared to a standard BST, the AVL Tree provides more consistent performance by preventing excessive tree height growth.
2. Route Optimization:
   Dijkstra's algorithm successfully identified the shortest collection route between selected city zones. This helps reduce travel distance and improves vehicle utilization efficiency for waste collection operations.
3. Service Request Management:
   The Queue data structure processed citizen requests in First-In-First-Out (FIFO) order, ensuring fair and systematic handling of complaints and service tickets.
4. Recovery Log Management:
   The Stack data structure maintained a history of system actions. The most recent actions could be reviewed or removed efficiently using O(1) push and pop operations.
5. Vehicle Identification:
   The Hash Table enabled fast vehicle lookup using registration numbers, improving asset tracking and reducing search time compared to sequential searching methods.

Conclusion

The Smart Waste Collection & Recycling Management System successfully demonstrates the practical application of Data Structures and Algorithms in solving municipal waste management problems. The project integrates sorting, searching, trees, hashing, graphs, stacks, and queues into a single system that supports waste collection planning, vehicle management, route optimization, service request processing, and environmental reporting.

The implementation highlights how efficient data organization and algorithm selection can improve operational performance and resource utilization in waste management systems.

Future Scope:
• Integration of real IoT sensor data for automatic bin fill-level updates.
• GPS-based vehicle tracking and live route monitoring.
• Traffic-aware route optimization using dynamic graph weights.
• Database integration for large-scale municipal deployments.
• Mobile application support for citizen service requests.

---

7. References

1. GitHub Repos
2. Youtube
3. GeeksforGeeks — *AVL Trees, Graph Traversals, and Hashing Collision Resolutions*.
4. Class Notes
5. Github Link: https://github.com/Ankitraj-17/DSA-Final